

OpenTofu sur Debian 13

Installation, configuration et déploiement de VMs Proxmox avec OpenTofu et le provider `bpg/proxmox`.

- OpenTofu ≥ 1.6
- Debian 13 Trixie
- bpg/proxmox ≥ 0.78

01 Prérequis

Environnement système et Proxmox requis

Système

- Debian 13 (Trixie) avec accès `sudo`
- Connexion Internet pour l'installation des paquets
- `curl`, `wget`, `gnupg` disponibles

Proxmox

- Proxmox VE fonctionnel et accessible (`https://172.17.5.102:8006/`)
- Un token API créé pour OpenTofu (voir section 02)
- Une clé SSH configurée pour l'accès root à Proxmox
- Des templates de VMs déjà créés sur Proxmox (IDs définis dans `variables.tf`)

Vérification des prérequis

```
bash

# Vérifier la version de Debian
cat /etc/os-release | grep -E "NAME|VERSION"

# Vérifier la connectivité vers Proxmox
curl -k https://172.17.5.102:8006/ -o /dev/null -w "%{http_code}" --max-time 5
```

02 Configuration de l'accès Proxmox

Création du token API

Créer le token API

Sur l'interface web Proxmox (`https://172.17.5.102:8006/`) :

- 1 Aller dans Datacenter → Permissions → API Tokens
- 2 Cliquer sur Add
- 3 Renseigner : User = `opentofu@pve`, Token ID = `tofu`, décocher Privilege Separation
- 4 Copier le token généré (affiché une seule fois)
- 5 Assigner les droits via Datacenter → Permissions → Add → User Permission : `Path: /` · `User: opentofu@pve` · `Role: PVEVMAdmin`

03 Installation d'OpenTofu

Via script officiel ou APT manuel

Méthode recommandée — Script officiel

```
bash

curl --proto '=https' --tlsv1.2 -fsSL https://get.opentofu.org/install-opentofu.sh \
| sudo sh -s -- --install-method deb
```

Méthode manuelle — Via APT

●bash

```
# 1. Installer les dépendances
sudo apt update
sudo apt install -y wget curl gnupg software-properties-common apt-transport-https

# 2. Créer le dossier keyrings
sudo install -m 0755 -d /etc/apt/keyrings

# 3. Importer la clé GPG OpenTofu
curl -fsSL https://packages.opentofu.org/opentofu/tofu/gpgkey \
  | gpg --dearmor \
  | sudo tee /etc/apt/keyrings/opentofu.gpg > /dev/null
sudo chmod a+r /etc/apt/keyrings/opentofu.gpg

# 4. Ajouter le dépôt APT
echo "deb [signed-by=/etc/apt/keyrings/opentofu.gpg] \
  https://packages.opentofu.org/opentofu/tofu/any/ any main" \
  | sudo tee /etc/apt/sources.list.d/opentofu.list

# 5. Installer OpenTofu
sudo apt update && sudo apt install -y opentofu
```

Vérification

●bash

```
tofu --version
# Attendu : OpenTofu v1.x.x (ou supérieur)
```

04 Structure du projet

Arborescence et création du répertoire

●bash

```
mkdir -p ~/opentofu-proxmox
cd ~/opentofu-proxmox
```

Arborescence finale attendue :

```
opentofu-proxmox/ ├── provider.tf # Configuration du provider Proxmox ├── variables.tf # OS, templates, paramètres communs ├──
main.tf # Ressource VM et outputs └── clean_state.sh # Script de nettoyage du state
```

05 Fichiers de configuration

Contenu et explication de chaque fichier

provider.tf — Connexion à Proxmox

Déclare le provider `bpg/proxmox` et configure l'authentification à l'instance Proxmox.

```
provider "proxmox" {  
  terraform {  
    required_providers {  
      proxmox = {  
        source = "bpg/proxmox"  
        version = ">= 0.78.0"  
      }  
    }  
  }  
}  
  
provider "proxmox" {  
  endpoint = "https://172.17.5.102:8006/"  
  api_token = "opentofu@pve!tofu=ff73be51-0d82-448a-9934-a0259993bc2e"  
  insecure = true # Accepte les certificats auto-signés Proxmox  
  
  ssh {  
    agent = true # Utilise le SSH agent  
    username = "root"  
  }  
}
```

⚠ Sécurité — token API en clair

En production, utilisez une variable d'environnement : `export PROXMOX_VE_API_TOKEN="opentofu@pve!tofu=<token>"` et supprimez la ligne `api_token` du provider.

variables.tf — Catalogue OS et paramètres

Centralise les types d'OS supportés, IDs des templates Proxmox, plages de VMIDs et paramètres communs.

```
variables.tf  
  
# Seule variable à modifier en ligne de commande  
# Exemple : -var='vm_deployments={"debian"=3,"ubuntu"=2}'  
variable "vm_deployments" {  
  description = "Map OS => nombre de VMs à créer"  
  type        = map(number)  
  default     = { "debian" = 3 }  
  validation {  
    condition = alltrue([for os, count in var.vm_deployments : contains(keys(var.os_config), os)])  
    error_message = "OS supportés : debian, ubuntu, windows."  
  }  
}  
  
# Catalogue OS – template + plage de VMIDs  
variable "os_config" {  
  type = map(object({ template_id = number, vm_id_base = number, is_windows = bool, boot_disk = string }))  
  default = {  
    debian = { template_id = 100, vm_id_base = 200, is_windows = false, boot_disk = "virtio0" }  
    ubuntu = { template_id = 106, vm_id_base = 300, is_windows = false, boot_disk = "scsi0" }  
    windows = { template_id = 107, vm_id_base = 400, is_windows = true, boot_disk = "ide0" }  
  }  
}  
  
variable "target_node" { type = string; default = "Proxmox-SP2" }  
variable "storage" { type = string; default = "local-lvm" }
```

Variables à adapter à votre environnement :

VARIABLE	VALEUR ACTUELLE	À MODIFIER SI...
os_config.debian.template_id	100	Votre template Debian a un autre VMID
os_config.ubuntu.template_id	106	Votre template Ubuntu a un autre VMID
os_config.windows.template_id	107	Votre template Windows a un autre VMID
target_node	Proxmox-SP2	Votre nœud Proxmox a un autre nom
storage	local-lvm	Vous utilisez un stockage différent

main.tf — Ressource VM et outputs

Logique de création des VMs par clonage de templates Proxmox, avec outputs récapitulatifs.

main.tf

```
locals {
  # Aplatit {"debian"=2,"ubuntu"=1} en liste d'objets VM
  vm_list = flatten([
    for os, count in var.vm_deployments : [
      for i in range(count) : {
        key      = "${os}-${format("%02d", i + 1)}"
        name     = "${os}-${format("%02d", i + 1)}"
        vm_id    = var.os_config[os].vm_id_base + i
        template_id = var.os_config[os].template_id
        is_windows = var.os_config[os].is_windows
        boot_disk = var.os_config[os].boot_disk
      }
    ]
  ])
  # Transforme en map indexée par key (requis par for each)
  vm_map = { for vm in local.vm_list : vm.key => vm }
}

resource "proxmox_virtual_environment_vm" "vm" {
  for_each = local.vm_map
  name     = each.value.name
  vm_id    = each.value.vm_id
  node_name = var.target_node

  clone {
    vm_id      = each.value.template_id
    full       = true
    datastore_id = var.storage
  }

  boot_order = [each.value.boot_disk]
}

output "deployed_vms" {
  description = "Récapitulatif des VMs créées"
  value = {
    for key, vm in proxmox_virtual_environment_vm.vm :
      key => { vm_id = vm.vm_id, name = vm.name, node = vm.node_name }
  }
}

output "vm_count_by_os" {
  description = "Nombre de VMs par OS"
  value       = var.vm_deployments
}
```

clean_state.sh — Nettoyage du state

Retire les VMs du state OpenTofu sans les supprimer sur Proxmox.

clean_state.sh

```
#!/bin/bash
VMS=$(tofu state list | grep 'proxmox_virtual_environment_vm.vm')

[ -z "$VMS" ] && echo "Aucune VM trouvée dans le state." && exit 0

echo "VMs qui vont être retirées du state :"
echo "$VMS"
read -p "Confirmer la suppression du state uniquement ? (y/n) " CONFIRM
[ "$CONFIRM" != "y" ] && echo "Annulé." && exit 0

while read -r vm; do
  echo "Suppression du state : $vm"
  tofu state rm "$vm"
done <<< "$VMS"

echo "Toutes les VMs ont été retirées du state."
echo "Les VMs existent toujours sur Proxmox."
```

bash

```
chmod +x ~/opentofu-proxmox/clean_state.sh
```

06 Initialisation et déploiement

Premières commandes OpenTofu

tofu init

```
bash

cd ~/opentofu-proxmox
tofu init
```

✓ **Sortie attendue**
Installing bpg/proxmox v0.x.x... · OpenTofu has been successfully initialized!

tofu plan — Prévisualiser

```
bash

# Plan avec valeurs par défaut (3 VMs Debian)
tofu plan
```

tofu apply — Déployer

```
bash

# Avec confirmation manuelle
tofu apply

# Sans confirmation (CI/CD, scripts)
tofu apply -auto-approve
```

07 Utilisation courante

Commandes du quotidien

Déployer des VMs par OS

```
bash

# 5 VMs Debian
tofu apply -var='vm_deployments={"debian"]=5}'

# 2 VMs Ubuntu
tofu apply -var='vm_deployments={"ubuntu"]=2}'

# Mix : 3 Debian + 2 Ubuntu + 1 Windows
tofu apply -var='vm_deployments={"debian"]=3,"ubuntu"]=2,"windows"]=1}'
```

Gestion du state

```
bash

# Lister les ressources
tofu state list

# Inspecter une ressource spécifique
tofu state show 'proxmox_virtual_environment_vm.vm["debian-01"]'

# Rafraichir depuis Proxmox
tofu refresh
```

Afficher les outputs

```
bash

tofu output           # Tous les outputs
tofu output deployed_vms  # VMs créées avec VMID et nœud
tofu output vm_count_by_os # Comptage par OS
```

Destruction

●bash

```
# Détruire toutes les VMs (supprime sur Proxmox)
tofu destroy

# Détruire une VM spécifique uniquement
tofu destroy -target='proxmox_virtual_environment_vm.vm["debian-01"]'
```

Fichier terraform.tfvars

Pour éviter de répéter les options `-var`, créez un fichier `terraform.tfvars` chargé automatiquement par OpenTofu :

●terraform.tfvars

```
vm_deployments = { "debian" = 3, "ubuntu" = 2 }
target_node     = "Proxmox-SP2"
storage         = "local-lvm"
```